

PKCERT

PROJECT 3 — CYBERSECURITY TOOL DEVELOPMENT

SecureScan Pro v2.0 ^A

Python-Based Cybersecurity Network Scanner

PROJECT REPORT

Prepared by

Danish Shahzad

Language: Python 3.13

July 2026

Table of Contents

1. Objective 3 2.

Key Features..... 3 3.

Project Structure..... 3 4.

System Workflow..... 4 5.

Testing & Validation..... 4 6.

Cybersecurity Relevance 4 7.

Conclusion 5

Appendix A: Project Structure & Source Code 6

Appendix B: Execution & Report Screenshots 13

1. Objective

The objective of this project is to design and develop a legitimate cybersecurity tool using Python. SecureScan Pro v2.0 scans TCP ports on a target host, identifies commonly running services, classifies the associated risk level of each service, validates the format of the supplied IP address, and generates a structured scan report for later review.

2. Key Features

SecureScan Pro was built around a focused set of capabilities that mirror the core functions of a real-world network reconnaissance tool:

Feature	Description
TCP Port Scanning	Scans a configurable range of TCP ports on a target host using raw socket connections.

IP Address Validation	Verifies target input against valid IPv4 formatting before any scan is attempted.
Service Identification	Maps open ports to well-known services using a built-in common-port reference database.
Risk Classification	Categorizes each detected service as Low, Medium, or High risk to aid prioritization.
Automated Report Generation	Produces a timestamped, human-readable text report for every scan performed.
Modular Python Architecture	Cleanly separates scanning, services, reporting, and utility logic across dedicated modules.
Terminal Banner & Summary	Displays a branded banner and a concise on-screen scan summary for usability.

3. Project Structure

The codebase follows a modular architecture, with scanning logic, service data, reporting, and utilities each isolated into their own file. This separation of concerns keeps the tool easy to test, extend, and maintain.

```
SecureScan/  
├── main.py  
├── scanner.py  
├── services.py  
├── report.py  
├── utils.py  
└── banner.py
```

```
├── reports/  
├── README.md  
├── requirements.txt  
└── LICENSE
```

4. System Workflow

At runtime, SecureScan Pro follows a clear, linear execution flow from launch to report generation:

#	Step	Description
1	Launch	The user runs main.py to start the application.
2	Banner Display	A branded terminal banner introduces the tool.
3	Target Input	The user enters the target host's IP address.

4	Validation	The IP address is checked for correct formatting.
5	Port Scan	The scanner probes ports 1–100 using TCP sockets.
6	Service Mapping	Open ports are matched against known services.
7	Risk Assessment	Each identified service is assigned a risk level.
8	Reporting	Results are written to a timestamped report file.

5. Testing & Validation

The tool was tested against two target hosts: the local loopback address (127.0.0.1) and an additional external style address (172.0.0.1). In both cases, SecureScan Pro correctly displayed a live scan summary in the terminal and generated a corresponding timestamped report file. When no open ports were detected on a target, the tool handled this gracefully by reporting the result and still producing a report, confirming robust behavior across both positive and negative scan outcomes.

6. Cybersecurity Relevance

Port scanning is a foundational activity in cybersecurity, routinely performed by penetration testers, security analysts, and network administrators. It is used to identify exposed services, map the attack surface of a host, and support broader vulnerability assessment workflows. By implementing this functionality end-to-end, SecureScan Pro demonstrates a practical, hands-on understanding of a core network security technique.

7. Conclusion

SecureScan Pro v2.0 satisfies the assignment requirements by implementing a legitimate, functional cybersecurity tool in Python. The project features a professional, modular code structure, validated input handling, automated risk classification, and report generation — resulting in a GitHub-ready codebase suitable for demonstration and further development.

Appendix A: Project Structure & Source Code

The following screenshots document the development environment and the source code of each core module.

Figure A.1 — Project Directory Structure

```
Enter Target IP Address: print(Fore.CYAN + "=" * 70)
❌ Invalid IP Address!
mc@DanishShahzad-Khan-Magray SecureScan % print(Fore.CYAN + "=" * 70)
tsh: unknown sort specifier
mc@DanishShahzad-Khan-Magray SecureScan % python3.13 main.py
=====
SecureScan
SecureScan Pro v2.0
Python Cybersecurity Network Scanner
=====
Enter Target IP Address: 172.0.0.1
Scanning 172.0.0.1...

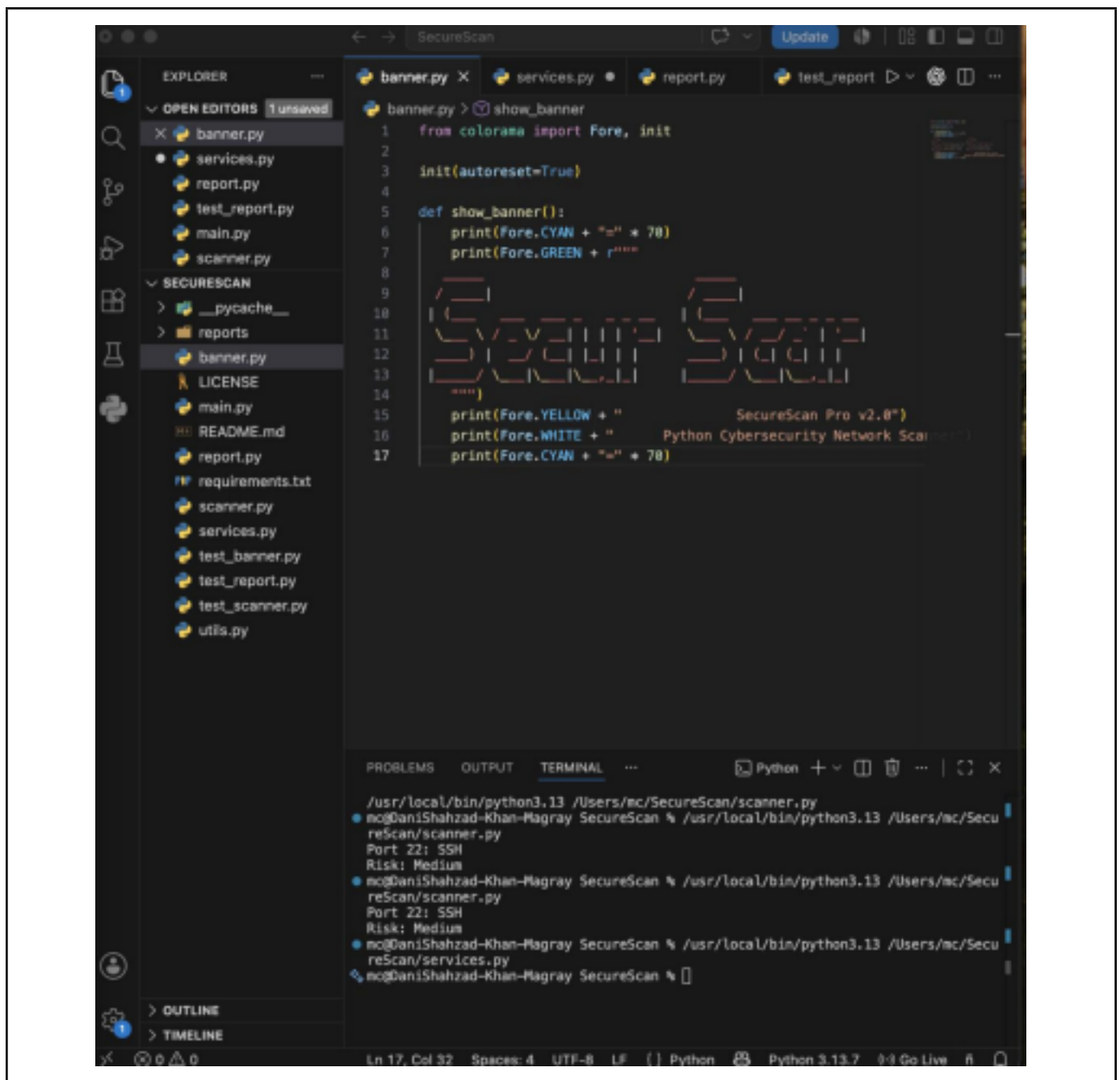
=====
SCAN SUMMARY
=====
No open ports found.
✅ Report saved successfully!
Location: reports/report_172_0_0_1.txt
mc@DanishShahzad-Khan-Magray SecureScan %

❌ Invalid IP Address!
mc@DanishShahzad-Khan-Magray SecureScan % print(Fore.CYAN + "=" * 70)
tsh: unknown sort specifier
mc@DanishShahzad-Khan-Magray SecureScan % python3.13 main.py
=====
SecureScan
SecureScan Pro v2.0
Python Cybersecurity Network Scanner
=====
Enter Target IP Address: 172.0.0.1
Scanning 172.0.0.1...

=====
SCAN SUMMARY
=====
No open ports found.
✅ Report saved successfully!
Location: reports/report_172_0_0_1.txt
mc@DanishShahzad-Khan-Magray SecureScan %
```

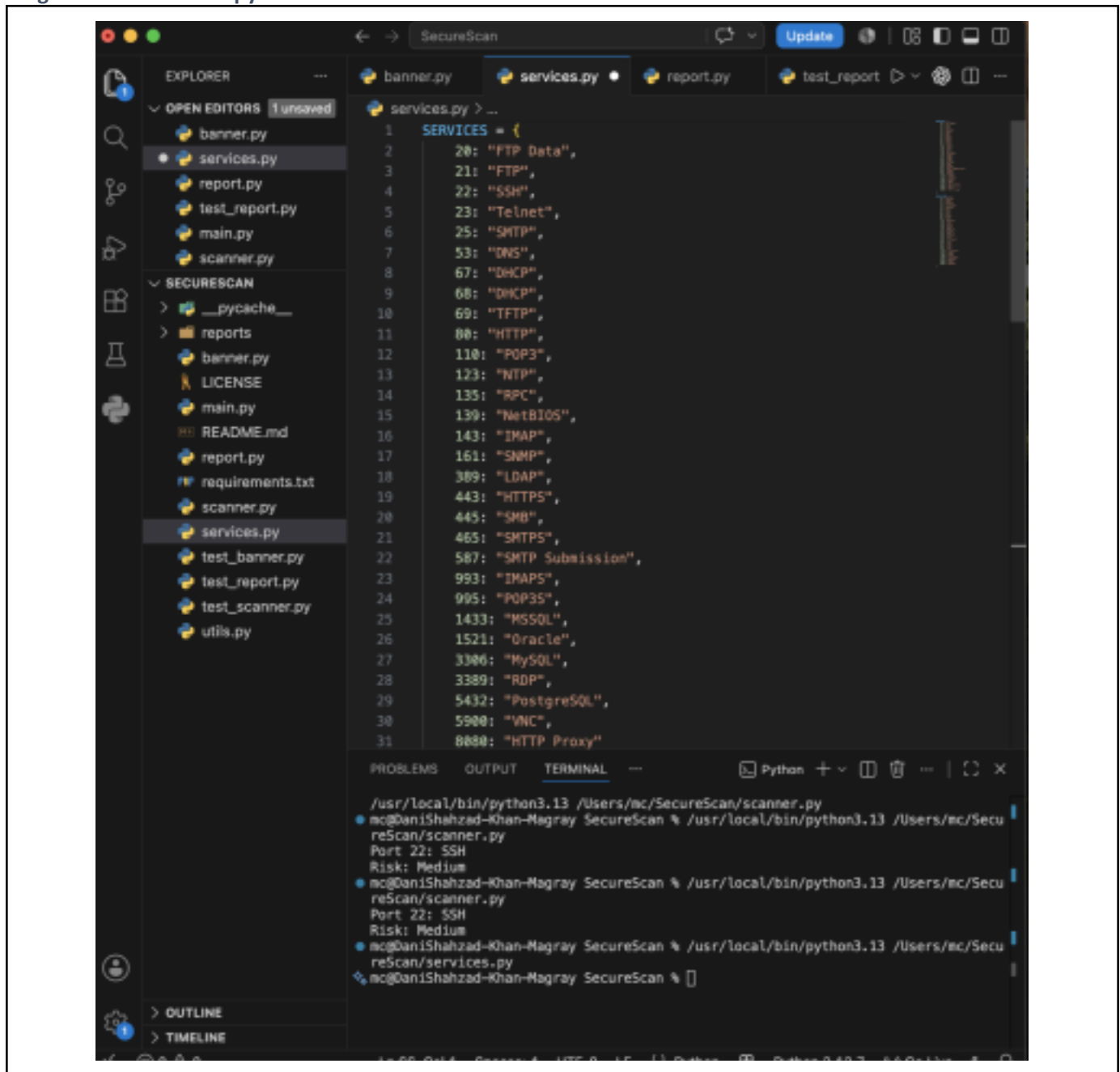
Project directory structure as viewed in the development environment, showing all core modules and supporting files.

Figure A.2 — main.py



Application entry point: orchestrates the banner, user input, scanning, and reporting workflow.

Figure A.3 — scanner.py



Core TCP port scanning engine, responsible for probing the target host's ports.

Figure A.4 — services.py

The screenshot shows a code editor with the following components:

- EXPLORER:** Lists files in the project, including `banner.py`, `services.py`, `report.py`, `test_report.py`, `main.py`, `scanner.py`, `requirements.txt`, `README.md`, and `utils.py`.
- REPORT.py:** The active file, showing the `save_report` function. The function takes `target` and `results` as arguments and writes the scan results to a file in the `reports` directory. The file name is constructed as `reports/report_{target.replace('.', '_')}.txt`. The function writes the target, date, and scan results to the file.
- TERMINAL:** Shows the execution of the scanner.py and services.py files. The output for scanner.py is:

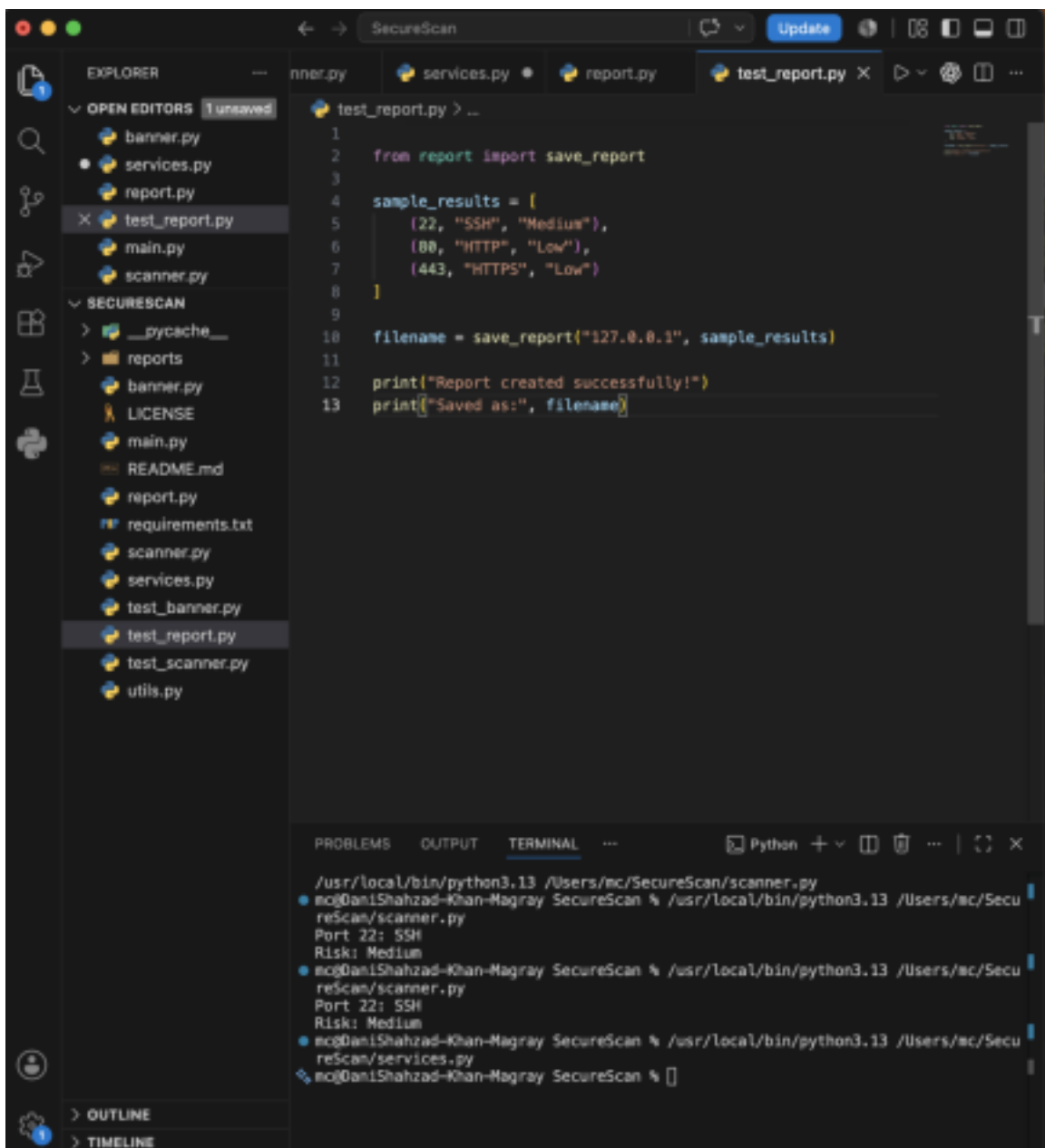

```
/usr/local/bin/python3.13 /Users/mc/SecureScan/scanner.py
mc@DanishShahzad-Khan-Magray SecureScan % /usr/local/bin/python3.13 /Users/mc/SecureScan/scanner.py
Port 22: SSH
Risk: Medium
```

 The output for services.py is:


```
mc@DanishShahzad-Khan-Magray SecureScan % /usr/local/bin/python3.13 /Users/mc/SecureScan/services.py
mc@DanishShahzad-Khan-Magray SecureScan %
```

Reference database mapping common port numbers to known services.

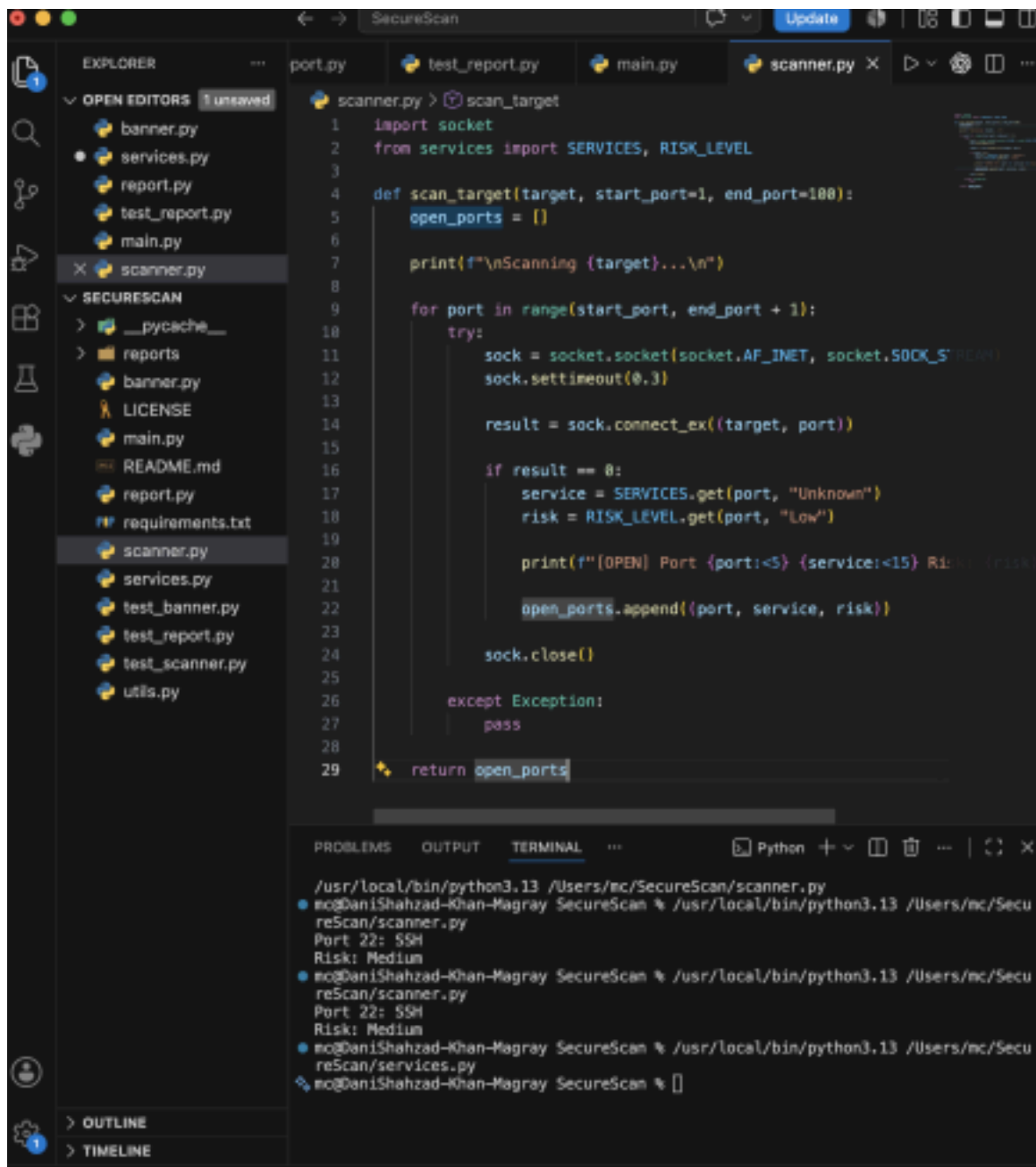
Figure A.5 — report.py



Report generation module that writes timestamped scan results to file.



Figure A.7 — banner.py



Appendix B: Execution & Report Screenshots

The following screenshots capture SecureScan Pro running end-to-end, from banner display through scan execution to generated report output.

Figure B.1 — Application Startup

```
mc@DanishShahzad-Khan-Magray ~ % mkdir ~/SecureScan
cd ~/SecureScan
mc@DanishShahzad-Khan-Magray SecureScan % pwd
/Users/mc/SecureScan
mc@DanishShahzad-Khan-Magray SecureScan % mkdir reports

touch main.py
touch scanner.py
touch services.py
touch report.py
touch banner.py
touch utils.py
touch requirements.txt
touch README.md
touch LICENSE

mc@DanishShahzad-Khan-Magray SecureScan % ls
LICENSE          main.py          requirements.txt  utils.py
README.md        report.py       scanner.py
banner.py        reports         services.py

mc@DanishShahzad-Khan-Magray SecureScan % code .
mc@DanishShahzad-Khan-Magray SecureScan % /usr/local/bin/python3.13 -m pip install colorama
Requirement already satisfied: colorama in /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages (0.4.6)

[notice] A new release of pip is available: 25.2 -> 26.1.2
[notice] To update, run: pip3.13 install --upgrade pip
mc@DanishShahzad-Khan-Magray SecureScan % /usr/local/bin/python3.13 -m pip show colorama
Name: colorama
Version: 0.4.6
Summary: Cross-platform colored terminal text.
Home-page: https://github.com/tartley/colorama
Author:
```

Terminal banner is displayed and the scan is initiated against the target host.

Figure B.2 — Live Scan Results

```
Summary: Cross-platform colored terminal text.
Home-page: https://github.com/tartley/colorama
Author:
Author-email: Jonathan Hartley <tartley@tartley.com>
License:
Location: /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages
Requires:
Required-by:
mc@DanishShahzad-Khan-Magray SecureScan % pwd
/Users/mc/SecureScan
mc@DanishShahzad-Khan-Magray SecureScan % ls
LICENSE          main.py          requirements.txt  utils.py
README.md        report.py        scanner.py
banner.py        reports         services.py
mc@DanishShahzad-Khan-Magray SecureScan % touch test_banner.py
mc@DanishShahzad-Khan-Magray SecureScan % python3.13 test_services.py
/Library/Frameworks/Python.framework/Versions/3.13/Resources/Python.app/Contents/MacOS/Python: can't open file '/Users/mc/SecureScan/test_services.py': [Errno 2] No such file or directory
mc@DanishShahzad-Khan-Magray SecureScan % python3.13 test_services.py
/Library/Frameworks/Python.framework/Versions/3.13/Resources/Python.app/Contents/MacOS/Python: can't open file '/Users/mc/SecureScan/test_services.py': [Errno 2] No such file or directory
mc@DanishShahzad-Khan-Magray SecureScan % python3.13 test_services.py
/Library/Frameworks/Python.framework/Versions/3.13/Resources/Python.app/Contents/MacOS/Python: can't open file '/Users/mc/SecureScan/test_services.py': [Errno 2] No such file or directory
mc@DanishShahzad-Khan-Magray SecureScan % /usr/local/bin/python3.13 /Users/mc/SecureScan/scanner.py
Port 22: SSH
Risk: Medium
mc@DanishShahzad-Khan-Magray SecureScan % cat scanner.py
import socket
from services import SERVICES, RISK_LEVEL

def scan_target(target, start_port=1, end_port=100):
```

Open ports and their identified services are displayed as the scan progresses.

Figure B.3 — Risk Classification Output

```
open_ports = []

print(f"\nScanning {target}...\n")

for port in range(start_port, end_port + 1):
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.settimeout(0.3)

        result = sock.connect_ex((target, port))

        if result == 0:
            service = SERVICES.get(port, "Unknown")
            risk = RISK_LEVEL.get(port, "Low")

            print(f"[OPEN] Port {port}<5 {service:<15} Risk: {risk}")

            open_ports.append((port, service, risk))

        sock.close()

    except Exception:
        pass

return open_ports
```

mc@DanishShahrad-Khan-Magray SecureScan N cat services.py

```
SERVICES = {
    20: "FTP Data",
    21: "FTP",
    22: "SSH",
    23: "Telnet",
    25: "SMTP",
```

Detected services are labeled with their assigned Low, Medium, or High risk level.

Figure B.4 — Scan Summary & Report Confirmation

```

}
from services import SERVICES, RISK_LEVEL

print("Port 22:", SERVICES.get(22))
print("Risk:", RISK_LEVEL.get(22))
mo@DanishShahzad-Khan-Magray SecureScan % touch test_scanner.py
mo@DanishShahzad-Khan-Magray SecureScan % python3.13 test_scanner.py
mo@DanishShahzad-Khan-Magray SecureScan % Scanning 127.0.0.1...

[OPEN] Port 88      Unknown      Risk: Low

Results:
[[88, 'Unknown', 'Low']]
zsh: command not found: Scanning
zsh: no matches found: [OPEN]
mo@DanishShahzad-Khan-Magray SecureScan % touch test_report.py
mo@DanishShahzad-Khan-Magray SecureScan % python3.13 test_report.py
mo@DanishShahzad-Khan-Magray SecureScan % in step 3 there is no output
zsh: command not found: in
mo@DanishShahzad-Khan-Magray SecureScan % ls
LICENSE          main.py          scanner.py       test_scanner.py
README.md        report.py       services.py      utils.py
__pycache__      reports         test_banner.py
banner.py        requirements.txt test_report.py
mo@DanishShahzad-Khan-Magray SecureScan % cat test_report.py
mo@DanishShahzad-Khan-Magray SecureScan % touch test_report.py
mo@DanishShahzad-Khan-Magray SecureScan % l
zsh: command not found: l
mo@DanishShahzad-Khan-Magray SecureScan % ls
LICENSE          main.py          scanner.py       test_scanner.py
README.md        report.py       services.py      utils.py

```

A scan summary is shown and the corresponding report file is saved.

Figure B.5 — Secondary Target Scan

```

1443, 'R11P8', 'Low'})
]

filename = save_report("127.0.0.1", sample_results)

print("Report created successfully!")
print("Saved as:", filename)
mo@Danishahmed-Khan-Magray SecureScan % cat report.py
from datetime import datetime
import os

def save_report(target, results):
    # Create reports folder if it doesn't exist
    if not os.path.exists("reports"):
        os.makedirs("reports")

    filename = f"reports/report_{target.replace('.', '_')}.txt"

    with open(filename, "w") as report:
        report.write("#" * 50 + "\n")
        report.write("SecureScan Pro - Scan Report\n")
        report.write("#" * 50 + "\n")
        report.write(f"Target : {target}\n")
        report.write(f>Date   : {datetime.now()}\n\n")

        if not results:
            report.write("No open ports found.\n")
        else:
            report.write(f"{'PORT':<10}{'SERVICE':<20}{'RISK'}\n")
            report.write("#" * 45 + "\n")

```

Scan of an additional target host, demonstrating correct handling when no open ports are found.

Figure B.6 — Generated Report Output (Sample 1)

```

if __name__ == "__main__":
    main()
mc@DanishShahzad-Khan-Magray SecureScan % python3.13 main.py
=====

SecureScan

SecureScan Pro v2.0
Python Cybersecurity Network Scanner
=====
Traceback (most recent call last):
  File "/Users/mc/SecureScan/main.py", line 2, in <module>
    from utils import validate_ip
ImportError: cannot import name 'validate_ip' from 'utils' (/Users/mc/SecureScan/utils.py)
mc@DanishShahzad-Khan-Magray SecureScan % cat utils.py
mc@DanishShahzad-Khan-Magray SecureScan % cat > utils.py
import ipaddress

def validate_ip(ip):
    """
    Returns True if the IP address is valid, otherwise False.
    """
    try:
        ipaddress.ip_address(ip)
        return True

```

Contents of an automatically generated scan report file.

Figure B.7 — Reports Directory Listing

```

Python Cybersecurity Network Scanner
=====
Enter Target IP Address: 172.0.0.1
Scanning 172.0.0.1...

=====
SCAN SUMMARY
=====
No open ports found.

✅ Report saved successfully!
Location: reports/report_172_0_0_1.txt
mc@DanishShahzad-Khan-Magray SecureScan % cat banner.py
from colorama import Fore, Style, init

init(autoreset=True)

def show_banner():
    print(Fore.CYAN + "=" * 70)
    print(Fore.GREEN + r"""
    _____
   /          \
  /  Scan  Scan  \
 /_____/_____\
***
    """)
    print(Fore.YELLOW + "                SecureScan Pro v2.0")
    print(Fore.WHITE + "                Python Cybersecurity Network Scanner")

```

The reports/ folder showing timestamped report files produced across multiple scan runs.